Documentation Abakub©

A detailed presentation of the KUBC machine model.

Robin Valmalette

05.06.2023

Contents

1 Préambule **2**

2 Présentation générale **3**

3 Utilisation normale **4**

3.1 Campagne 5

3.2 Lancement 6

4 Premier niveau d'explication : idée générale **6**

5 Création des inp (deuxième niveau) **8**

5.1 inp de l'essai principal (ligne de la campagne) 8

5.2 inp des essais complémentaires (PBC et/ou KUBC) 8

6 Résolution (deuxième niveau) **9**

7 Extraction des données (deuxième niveau) **9**

8 Computation des données (deuxième niveau) **9**

9 Regroupement des résultats **10**

1 Préambule

Voici quelques définitions de noms utilisés dans cette documentation :

- `inp` : input file (fichier Abaqus). Ce fichier suffit à Abaqus à lancer un calcul. Il contient tous les nœuds, conditions aux limites, chargements, TOUT pour une simulation. Pour lancer la simulation de d'un inp, il suffit de lancer une commande comme `abaqus_2021 job=inp_file`
- `odb` : output database (fichier Abaqus). Contient tous les résultats de la simulation.
- tmp = temporaire

Abaqus génère beaucoup d'autres fichiers mais qui ne nous intéresse pas ici (si tout se passe bien).

Dans ce rapport le *premier niveau* concerne les scripts en surface sans rentrer dans les détails. A ce niveau, les scripts sont en bash ou python3. On y trouve par exemple, l'ordre dans lequel on exécute la génération des inp, le post-traitement ou encore la suppression des fichiers intermédiaires. Le *deuxième niveau* rentre un peu plus en détail avec quelques scripts en python2 exécutés par Abaqus comme par exemple l'extraction des résultats. Le *troisième niveau* concerne vraiment le cœur le plus souvent en python2 pour Abaqus. Ce dernier niveau est de loin le moins digeste avec beaucoup de fonctions internes Abaqus.

Tous les fichiers python ne sont pas qu'en python3 mais aussi en python2. Si le script contient

```
from abaqus import *
from abaqusConstants import *
from caeModules import *
from driverUtils import executeOnCaeStartup
```

alors il s'agit d'un script pour Abaqus. Le mieux selon moi, c'est d'utiliser vsc (ou autre) pour localiser la source de la fonction (ctrl+clique avec vsc).

Je conseil VSC avec les extensions basiques suivantes : - abaqus - better comments - jupyter - log file highlighter - python

Pour avoir des meilleurs logs, modifier log file highlighter avec les occurrences suivantes peut être judicieux:

```
"abaqus_2021 job", "abaqus_2021 cae", "COMMAND", "TEST", "INFO", "
    __main__", "COMPLETED", "termination", "scp", "main_extract.py", "
    main_auto_compute.py", "error", "full_runner.py", "\\make.*", "START
    "
```

2 Présentation générale

Abakub© est un ensemble de scripts permettant de simuler des éprouvettes dans un cadre PBC, KUBC et KUBC discret (machine). Il permet de faire la génération des inp, la résolution par abaqus, et le post-traitement avec sauvegarde de chaque simulation.

L'arbre avec les fichiers principaux est donné ci-dessous.

```
.
|-- campagnes_vierges
|   |-- ...
|-- campagne_test
|   |-- odb
|   `-- results
|-- database
|   |-- ...
|-- KUBC_support_analysis
|   |-- ...
|-- porosity
|   |-- ...
|-- sample_bank
|   |-- ...
|-- src
|   |-- func
|   |-- geometry
|   |-- model_generation
|   |-- post_compute
|   |-- post_extract
|   |-- ressources
|   |-- th_extract
|   |-- th_gen
|   |-- constants.py
|   |-- debug.py
|   |-- field_heterogeneity.py
|   |-- main_compute.py
|   |-- main_extract.py
|   |-- make_inp.py
|   |-- make_job.py
|   |-- post_extract.py
|   |-- runner.py
|   `-- update_campaign.py
|-- theory_matrix_bank
|   |-- ...
|-- utilities
|   |-- ...
|-- 001_log.log
|-- campagne_test.csv
|-- clean.py (OSEF)
|-- documentation.md (OSEF)
|-- documentation.pdf
```

```
|-- eisvogel.latex (OSEF)
|-- from_scratch.py (OSEF)
|-- launcher.slurm
|-- README.md
|-- sub_run_job.sh
`-- sub_run_main.sh
```

- `src` contient l'ensemble des scripts nécessaires aux calculs.
- `utilities` contient l'ensemble de scripts du stage sur le traitement des résultats (exploitation de la database).
- `campagnes_vierges` est le répertoire de sortie de `make_campaign.py` qui sert à générer les grandes campagnes.
- `theory_matrix_bank` contient l'ensemble des matrices PBC et KUBC qui ont déjà été calculées pour éviter de refaire tourner les calculs. (peut être supprimé pour raz)
- `sample_bank` contient l'ensemble des géométries pour éviter de regénérer celles qui ont déjà été générées. (peut être supprimé pour raz)
- `porosity` contient toutes les porosités des cas simulés. (peut être supprimé pour raz)
- `KUBC_support_analysis` contient les scripts sur l'analyse des connections avec les supports (cf Martin)
- `campagne_test` contient les résultats de la campagne `campagne_test.csv`
- `database` contient les résultats des campagnes de manière condensée sans les images (très pratique à partager)

`ROOT` est la racine `..`

3 Utilisation normale

D'un point de vue purement utilisateur, nous créons une campagne (liste de tests) sous forme de tableau dans un `.csv` avec tous les paramètres. On renseigne ensuite dans le `launcher.slurm` quelle est la campagne à faire tourner avec le nombre de processus en parallèles. Enfin, on exécute le fichier `launcher.slurm`.

On notera qu'il est possible de ne pas passer par le slurm et d'exécuter seulement la commande suivant dans un terminal:

```
python3 src/runner.py <n_task> <ma_campagne_sans_exentsion_csv>
```

Une fois terminé, un dossier contenant tout les résultats sera présent à la racine, au même endroit que la campagne.

Le fichier csv de la campagne est modifié en remplaçant le bool de `flag_inp` par `False` si le test a correctement été complété.

3.1 Campagne

Créer une campagne et la placer dans le `ROOT`. Par exemple : `campagne_test.csv` avec `;` comme séparateur. La première ligne est réservée à la définition des paramètres :

- `debug` : Force la génération de l'éprouvette même si elle a été trouvée dans `sample_bank`. (bool)
- `test_id` : Identifiant unique de l'essai. C'est mieux de le rendre unique même avec toutes campagnes confondues. (str)
- `sample_name` : Nom de la géométrie (méso structure). La liste dispo se trouve dans `src/geometry`. (str avec lettre, chiffres et _)
- `dim` : modélisation 2D si vide, 3D si = `3.0`
- `geo_parameters` : paramètres géo de la géométrie. Cf `src/geometry` pour voir à quoi ils correspondent. (n float séparés par des virgules)
- `mesh_size` : taille du maillage (tous les éléments ont la même taille). (float)
- `thickness` : épaisseur de l'éprouvette (2 et 3D). (flaot)
- `trans_rot` : Translation et rotation du motif en [mm]. (3 float séparés par des virgules)
- `corner_cleaner` : Effectuer les entailles dans les coins. (bool, vide=`False`)
- `mat` : Module d'Young et coefficient de Poisson du matériau élastique. (2 float séparés par des virgules)
- `free_rotation` : Libre rotation ou non des cylindres. (bool)
- `free_u3` : Pour calcul 3D seulement : si `False` il y a adhérence parfaite entre l'éprouvette et les cylindres (déplacement hors plan bloqué). Si `True`, ça revient à faire du 2D. (bool)
- `perfect_panto` : Si `True` les cylindres sont supposés parfait. (bool)
- `stiff_panto` : si `perfect_panto=True` rentrer les défauts de souplesses. Dans l'ordre, cotés haut-bas, puis droit-gauche en [N/mm]. (2 float séparés par des virgules)
- `friction` : Dans l'ordre : coefficient de frottement, moyenne de la précharge (type palier), écart-type de la précharge, minimum de la précharge, maximum de la précharge.
- `clearance` : Dans l'ordre : Moyenne du jeu, écart-type du jeu, minimum du jeu, maximum du jeu.
- `load_1` : Etape de chargement 1. l'ordre : ϵ_{11},ϵ_{22} et γ_{12} (Attention, ce n'est pas ϵ_{12})
- `load_2` : idem
- `load_3` : idem. De mémoire on peut rajouter autant de chargement qu'on veut, le calcul tournera, le post-traitement peut-être pas…
- `flag_make_inp` : `True`, le test sera effectué au prochain lancement de la campagne, sinon non. Ça permet de lancer la campagne même si on veut faire qu'un seul test. (bool)
- `keep_odb` : Permet de garder l'odb d'Abaqus (fichier résultat). (bool)
- `status` : de mémoire, cette variable n'est plus utilisée.

- `comment` : Colone non utilisée par les scripts, destinée à l'utilisateur.
- `err` : paramètre devenu obsolète

Chaque paramètre sera interprété la fonction `eval` de python. Si cela échoue, le paramètre est considéré comme un `str`. Par exemple - True : donnera `True` le booléen - 56.3,2.6,1 : donnera le tuple `(56.3,2.6,1)` - honey : donnera le str `'honey`

Les paramètres sont rangés dans un dictionnaire donc l'ordre des colonnes n'a aucune importance. L'ordre des lignes définie *plus ou moins* l'ordre de calcul.

D'une manière plus générale, il suffit de rechercher le paramètre dans `src` pour voir son utilisation.

3.2 Lancement

Le fichier à exécuter est `lancher.slurm` avec slurm depuis ~. Par exemple, le root est dans le `$USRTMP` (`/usrtmp/valmalette/abakubc`), alors dans `/u/valmalette`, il faut exécuter sbatch `sbatch $USRTMP/abakubc/launcher.slurm`.

Dans `lancher.slurm`, `ntasks` correspond au nombre de calcul en parallèle + 1 (donc minimum 2), adapter le répertoire à la ligne `cd /usrtmp/valmalette/abakubc/`. Donner le nom de la campagne à la ligne suivant (sans le `.csv`).

4 Premier niveau d'explication : idée générale

Le script de plus haut niveau est `lancher.slurm` qui permet de lancer l'ensemble du programme en slurm. Il ne fait qu'exécuter le script python `src/runner.py` qui pilote TOUT. `src/runner.py` gère en effet la parallélisation, la génération des inp, le lancement de la résolution par Abaqus, l'extraction, le post-traitement, et le nettoyage des fichiers/répertoires intermédiaires. Regardons ce que `src/runner.py` fait en détail.

Au début, la campagne est lue et on détermine les tests qui doivent être lancés grâce au paramètre `flag_inp` dans la campagne. Puis on fait tourner la fonction `run_test` pour chaque test à effectuer grâce à :

```
1 with Pool(processes=N_TASK-1) as pool:
2     results = pool.imap_unordered(run_test, tests_to_run)
```

Ici, on exécute la fonction `run_test` avec chaque item de la liste `tests_to_run` comme argument de la fonction. Chaque item étant un dictionnaire avec tous les paramètres de l'essai. `N_TASK-1` processus sont créés et chaque processus exécute la fonction `run_test` avec le premier item de la

liste `tests_to_run` qui n'a pas encore été lancé ou terminé par un processus. On notera que les essais ne sont pas nécessairement faits dans l'ordre. Chaque processus a un log situé dans le root qui permet de suivre sont avancement.

Pour chaque essai, un répertoire **tmp** est créé et nommé par le test_id (cf campagne). C'est dans ce répertoire que la génération de l'inp, le calcul par Abaqus et l'extraction sont faits. En effet, avec les processus en parallèles, on ne peut pas avoir plusieurs Abaqus qui tournent en même temps dans le même répertoire. Afin de pouvoir lancer Abaqus dans ces répertoires, on utilise les scripts `sub_run_job.sh` et `sub_run_main.sh` avec des arguments.

Ensuite, on commence le dur du travail avec la génération de l'inp principal qui est l'essai de la ligne de la campagne (essai dans la machine). Cela est réalisé avec le script `src/make_inp.py` via :

```
check_call(['./sub_run_main.sh', test['test_id'], 'make_inp', str(test)
    .replace(' ', ''), str(paths).replace(' ', '')])
```

Puis, si besoin, on génère un inp pour le PBC et KUBC théoriques avec le même style de commande. Dans certains cas, ce n'est pas utile : - si la méso-structure est monolithique ou type Penrose vis à vis du PBC. Le détail de celle liste peut être modifiée dans `src/constants.py` (variables `model_with_KUBC_th` et `model_with_PBC_th`). - si le calcul PBC et/ou KUBC a déjà été calculé lors d'une campagne précédente ou par un processus plus tôt dans la campagne. - si un autre processus a déjà commencé la génération ou la résolution mais n'a pas encore fini d'extraire les données.

Puis, chaque inp est soumis à Abaqus (submit the jobs) avec :

```
check_call(['./sub_run_job.sh']+jobs_list)
```

Après avoir fait tourner toutes les simulations, on extrait les données importantes avec la ligne

```
main_extract.proceed_extract_bis(paths, test, KUBC_name, PBC_name)
```

J'utilise le terme *extraction* pour tout ce qui relève de l'exploitation de l'odb car ce sont des opérations en script Abaqus (et donc en python2). Dans un second temps seulement, on fait la *computation* des résultats (en python3 sans Abaqus, l'odb n'est plus utile) avec la ligne :

```
main_compute.proceed_computation(paths, test)
```

Enfin, on déplace le fichier odb dans le dossier de la campagne si le paramètre `keep_odb` de la campagne vaut `True`, sinon on le supprime. On supprime le répertoire tmp et on passe à l'essai suivant.

5 Création des inp (deuxième niveau)

5.1 inp de l'essai principal (ligne de la campagne)

Par *test principale*, j'entends l'essais de la ligne et pas le PBC ou KUBC théorique. `src/make_inp.py` génère cet inp, c'est ce que nous allons expliquer ici. Ce script est en python2 car il est exécuté par Abaqus sous forme de script (attention, certaines commandes ne fonctionnent pas et ne donne pas d'erreur très explicite, par exemple, les f-string n'existent pas !!!!!)

On commence par traduire les arguments d'entré avec la création d'un dictionnaire `test` avec toutes les infos du test données par la campagne grâce à :

```
test = eval(sys.argv[-2])
for i in test:
    if len(test[i])==0:
        test[i]=None
    else:
        try:
            test[i] = eval(test[i]) if i not in ['sample_name', 'status
                ', 'comment', 'err'] else test[i]
        except NameError:
            pass
```

Ce bloque de code revient souvent dans les scripts pour avoir accès rapidement et de manière explicite aux paramètres de l'essais. Par exemple, `test['debug']` vaut soit `True` ou `False` (bool python).

Si la géométrie n'a pas été trouvée dans la banque de géométries, alors on génère l'éprouvette :

```
if '{}.sat'.format(geometry_name) not in os.listdir(paths[
    SAMPLE_BANK_DIR]) or FORCE_GEN:
```

Dans cette boucle, on génère l'éprouvette, on effectue les entailles si besoin et on sauvegarde la géométrie et la liste des supports manquants.

Ensuite, on initialise la liste des étapes de chargement et on procède à l'assemblage avec la ligne :

```
make_assembly.main_assemble_model(test, paths, step_list)
```

Le script `make_assembly` est du pur abaqus et n'est pas très digeste avec beaucoup de fonction interne (voir niveau trois)

5.2 inp des essais complémentaires (PBC et/ou KUBC)

La génération de ces inp est beaucoup plus simple et est effectuée par les scripts dans `src/th_gen/`. A ce stade, il y a un script par méso-structure car le gros du script consiste en la génération de

l'éprouvette.

6 Résolution (deuxième niveau)

Le lancement de la résolution est finalement assez simple avec le script `sub_run_job.sh`. C'est dans ce fichier qu'on **change la mémoire allouée par calcul**.

7 Extraction des données (deuxième niveau)

La partie d'extraction vise à sauvegarder toutes les données/champs des odb pour pouvoir ensuite s'en passer. La fonction `main_extract.proceed_extract_bis` est la fonction principale.

Si le processus était bien chargé de faire un KUBC ou PBC, il extrait les données associées avec

```
check_call(['./sub_run_main.sh', test['test_id'], f'th_extract/
    extract_th', str(test).replace(' ', ''), th_KUBC_name])
```

Ensuite, **si** la solution théorique est nécessaire et n'est pas dans `theory_matrix_bank`, et qu'en plus qu'un fichier `f'{KUBC_name}.tmp'` existe indiquant qu'un autre processus est en train de le faire, **alors** on attend:

```
if test['sample_name'] in model_with_KUBC_th:
    logging.info(f'Waiting for KUBC th')
    while (f'{KUBC_name}.json' not in os.listdir(paths[THEORY_DIR])):
        sleep(1)
        if f'{KUBC_name}.tmp' not in os.listdir(paths[ROOT]):
            raise FileNotFoundError
```

Une fois que les solutions théoriques sont obtenues, alors on peut faire l'extraction de l'essai principal avec `src/post_extract.py`.

8 Computation des données (deuxième niveau)

Si l'extraction se focalisait si l'extraction des champs de déformation, de contrainte, des forces dans les cylindres, etc., la computation elle calcule :

- les champs de contraintes moyens via les efforts dans les pantos
- les champs de contraintes moyens via les contraintes dans les éléments et la porosité,
- les champs de déformations moyen via le déplacement de chaque nœud,
- les raideurs avec les différents champs,

- les MAUVAISES ERREURS ATTENTION, NE PAS UTILISER glob_err. Ce pb toujours présent est finalement corrigé lors de l'exploitation de la database.

Toutes les données sont sauvegardées dans le répertoire de la campagne dans le fichier `final_data.json`, par exemple `campagne_test/results/Z_test_1/final_data.json`.

9 Regroupement des résultats

Le script `utilities/compile_result_in_one_file.py` permet de regrouper tous les `final_data.json` dans un seul `.json` et toutes les images dans le `.iii`. Tout est enregistré dans `database`.

Dans le `.iii`, toutes les images sont regoupée à la chaine (en binaire) et séparée par le `test_id`.